Foundation Implementation

This implementation section brings together all Foundation phase specifications into a complete, working foundation for your chatbot application. You'll implement the project structure, advanced Kiro features, validated requirements, and system architecture.

Implementation Overview

Based on your Foundation specifications, you'll implement:

- **Complete project structure** with professional tooling and configuration
- **Kiro integration** with MCP servers, Agent Hooks, and Agent Steering
- **Requirements validation** with stakeholder alignment and documentation
- **System architecture** with scalable, secure, and maintainable design

Step 1: Project Structure Implementation

Implement the complete project structure as specified:

~~~
Create the comprehensive project structure:

```bash
# Create main project structure
mkdir -p MyCoolChatbot-KiroSpec/{src,docs,tests,scripts,config}
cd MyCoolChatbot-KiroSpec

# Initialize project with proper configuration
npm init -y
git init

# Create detailed directory structure
mkdir -p src/{components,services,utils,types,hooks,contexts}
mkdir -p src/components/{ui,forms,layout,chat}
mkdir -p src/services/{api,auth,ai,storage}
mkdir -p tests/{unit,integration,e2e}
mkdir -p docs/{api,architecture,deployment,user}
mkdir -p config/{environments,security,monitoring}
mkdir -p scripts/{build,deploy,maintenance}

# Create Kiro configuration directory
mkdir -p .kiro/{settings,steering,hooks}

# Create essential configuration files
touch .env.example .env.local
touch .gitignore .gitattributes
touch README.md CONTRIBUTING.md SECURITY.md
touch package.json tsconfig.json
touch jest.config.js playwright.config.ts
touch tailwind.config.js postcss.config.js
touch .eslintrc.js .prettierrc
```

**Package.json Configuration**:
```json
{
  "name": "mycoolchatbot-kirospec",
  "version": "1.0.0",
  "description": "Production-ready MyCoolChatbot-KiroSpec application with AI capabilities",
  "main": "src/index.tsx",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "test": "jest",
    "test:watch": "jest --watch",
    "test:coverage": "jest --coverage",
    "test:e2e": "playwright test",
    "lint": "eslint src --ext .ts,.tsx",
    "lint:fix": "eslint src --ext .ts,.tsx --fix",
    "format": "prettier --write src/**/*.{ts,tsx}",
    "type-check": "tsc --noEmit",
    "security:audit": "npm audit",
    "security:scan": "npm run lint && npm run type-check",
    "build:analyze": "npm run build && npx vite-bundle-analyzer",
    "docs:generate": "typedoc src --out docs/api",
    "prepare": "husky install"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.8.0",
    "zustand": "^4.3.0",
    "axios": "^1.3.0",
    "@aws-sdk/client-bedrock-runtime": "^3.0.0",
    "lucide-react": "^0.300.0",
    "clsx": "^1.2.0",
    "tailwind-merge": "^1.10.0"
  },
  "devDependencies": {
    "@types/react": "^18.0.0",
    "@types/react-dom": "^18.0.0",
    "@types/node": "^18.0.0",
    "@typescript-eslint/eslint-plugin": "^5.0.0",
    "@typescript-eslint/parser": "^5.0.0",
    "@vitejs/plugin-react": "^3.0.0",
    "typescript": "^4.9.0",
    "vite": "^4.0.0",
    "jest": "^29.0.0",
    "@testing-library/react": "^13.0.0",
    "@testing-library/jest-dom": "^5.0.0",
    "playwright": "^1.30.0",
    "eslint": "^8.0.0",
    "prettier": "^2.8.0",
    "husky": "^8.0.0",
    "lint-staged": "^13.0.0",
    "tailwindcss": "^3.2.0",
    "autoprefixer": "^10.4.0",
    "postcss": "^8.4.0"
  }
}
```

**TypeScript Configuration**:
```json
{
  "compilerOptions": {
    "target": "ES2020",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "module": "ESNext",
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true,
    "baseUrl": ".",
    "paths": {
      "@/*": ["./src/*"],
      "@/components/*": ["./src/components/*"],
      "@/services/*": ["./src/services/*"],
      "@/utils/*": ["./src/utils/*"],
      "@/types/*": ["./src/types/*"],
      "@/hooks/*": ["./src/hooks/*"]
    }
  },
  "include": ["src"],
  "references": [{ "path": "./tsconfig.node.json" }]
}
```
~~~

Step 2: Kiro Features Implementation

Implement comprehensive Kiro integration as specified:

~~~
Set up MCP servers, Agent Hooks, and Agent Steering:

**MCP Configuration (.kiro/settings/mcp.json)**:
```json
{
  "mcpServers": {
    "aws-docs": {
      "command": "uvx",
      "args": ["awslabs.aws-documentation-mcp-server@latest"],
      "env": {
        "FASTMCP_LOG_LEVEL": "ERROR"
      },
      "disabled": false,
      "autoApprove": ["search-documentation"],
      "disabledTools": []
    },
    "github": {
      "command": "uvx",
      "args": ["github-mcp-server@latest"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
      },
      "disabled": false,
      "autoApprove": ["search-repositories", "get-file-contents"],
      "disabledTools": []
    },
    "web-search": {
      "command": "uvx",
      "args": ["web-search-mcp-server@latest"],
      "env": {
        "SEARCH_API_KEY": "${SEARCH_API_KEY}"
      },
      "disabled": false,
      "autoApprove": ["web-search"],
      "disabledTools": []
    },
    "npm-registry": {
      "command": "uvx",
      "args": ["npm-registry-mcp-server@latest"],
      "disabled": false,
      "autoApprove": ["search-packages", "get-package-info"],
      "disabledTools": []
    }
  }
}
```

**Agent Steering - Project Standards (.kiro/steering/project-standards.md)**:
```markdown
---
inclusion: always
---

# Project Standards and Guidelines

## Code Quality Standards
- Use TypeScript for all source code with strict type checking
- Follow React best practices with functional components and hooks
- Implement comprehensive error handling and logging
- Maintain test coverage above 80% for all critical paths
- Use ESLint and Prettier for consistent code formatting

## Security Requirements
- Validate all user inputs and sanitize outputs
- Implement proper authentication and authorization
- Use HTTPS for all communications
- Follow OWASP security guidelines
- Regular security audits and dependency updates

## Performance Standards
- Target Core Web Vitals: LCP < 2.5s, FID < 100ms, CLS < 0.1
- Implement code splitting and lazy loading
- Optimize bundle size and minimize dependencies
- Use efficient state management patterns
- Implement proper caching strategies

## Accessibility Requirements
- Follow WCAG 2.1 AA guidelines
- Implement proper ARIA labels and roles
- Ensure keyboard navigation support
- Maintain color contrast ratios
- Test with screen readers

## Documentation Standards
- Maintain comprehensive README with setup instructions
- Document all APIs with OpenAPI specifications
- Include inline code comments for complex logic
- Keep architecture documentation current
- Provide user guides and troubleshooting information
```

**Agent Steering - Security Requirements (.kiro/steering/security-requirements.md)**:
```markdown
---
inclusion: fileMatch
fileMatchPattern: "**/*.{ts,tsx,js,jsx}"
---

# Security Implementation Requirements

## Authentication and Authorization
- Implement JWT-based authentication with proper expiration
- Use secure session management with httpOnly cookies
- Implement role-based access control (RBAC)
- Support multi-factor authentication (MFA)
- Validate all authentication tokens server-side

## Data Protection
- Encrypt sensitive data at rest and in transit
- Implement proper key management and rotation
- Follow data minimization principles
- Implement secure data deletion procedures
- Maintain audit trails for all data access

## Input Validation and Output Encoding
- Validate all user inputs on both client and server
- Implement proper output encoding to prevent XSS
- Use parameterized queries to prevent SQL injection
- Sanitize file uploads and validate file types
- Implement rate limiting and request throttling

## Security Headers and Configuration
- Implement Content Security Policy (CSP)
- Use security headers (HSTS, X-Frame-Options, etc.)
- Configure secure CORS policies
- Implement proper error handling without information leakage
- Use secure communication protocols (TLS 1.3+)
```

**Agent Hook - Code Quality Validation (.kiro/hooks/code-quality-hook.json)**:
```json
{
  "name": "Code Quality Validation",
  "description": "Validates code quality on file save",
  "trigger": {
    "type": "file-save",
    "filePattern": "**/*.{ts,tsx,js,jsx}"
  },
  "actions": [
    {
      "type": "run-command",
      "command": "npm run lint -- ${filePath}",
      "description": "Run ESLint on saved file"
    },
    {
      "type": "run-command",
      "command": "npm run type-check",
      "description": "Validate TypeScript types"
    },
    {
      "type": "agent-execution",
      "prompt": "Review the saved file for security vulnerabilities, performance issues, and adherence to project standards. Provide specific recommendations fo
      "context": ["${filePath}"]
    }
  ]
}
```

**Agent Hook - Test Automation (.kiro/hooks/test-automation-hook.json)**:
```json
{
  "name": "Automated Testing",
  "description": "Runs relevant tests when code changes",
  "trigger": {
    "type": "file-save",
    "filePattern": "src/**/*.{ts,tsx}"
  },
  "actions": [
    {
      "type": "run-command",
      "command": "npm run test -- --findRelatedTests ${filePath}",
      "description": "Run tests related to changed file"
    },
    {
      "type": "agent-execution",
      "prompt": "Analyze the test results and suggest additional test cases if coverage is insufficient. Ensure all edge cases are covered.",
      "context": ["${filePath}", "test-results"]
    }
  ]
}
```
~~~

Step 3: Requirements Implementation and Validation

Implement comprehensive requirements validation:

~~~
Create requirements validation and documentation system:

Requirements Documentation (docs/requirements/functional-requirements.md):
```markdown
# Functional Requirements Specification

## FR-001: User Authentication
**Priority**: High
**Status**: Specified
**Validation**: Automated testing required

### Description
Users must be able to securely authenticate using email/password with optional MFA.

### Acceptance Criteria
- [ ] User can register with email and password
- [ ] User can login with valid credentials
- [ ] User can enable/disable MFA
- [ ] Invalid login attempts are properly handled
- [ ] Session management follows security best practices

### Implementation Notes
- Use JWT tokens with proper expiration
- Implement secure password hashing (bcrypt)
- Support TOTP-based MFA
- Maintain audit logs for authentication events

## FR-002: Chat Interface
**Priority**: High
**Status**: Specified
**Validation**: User acceptance testing required

### Description
Users must be able to interact with the AI chatbot through an intuitive interface.

### Acceptance Criteria
- [ ] Real-time message exchange
- [ ] Message history persistence
- [ ] Typing indicators and status updates
- [ ] File upload capabilities
- [ ] Responsive design for all devices

### Implementation Notes
- Use WebSocket for real-time communication
- Implement message queuing for reliability
- Support markdown rendering in messages
- Optimize for accessibility (WCAG 2.1 AA)

## FR-003: AI Integration
**Priority**: High
**Status**: Specified
**Validation**: Performance testing required

### Description
The system must integrate with Amazon Bedrock for AI-powered responses.

### Acceptance Criteria
- [ ] Integration with Claude AI model
- [ ] Response time under 2 seconds
- [ ] Context awareness across conversation
- [ ] Error handling for AI service failures
- [ ] Content filtering and safety measures

### Implementation Notes
- Use AWS SDK for Bedrock integration
- Implement proper error handling and fallbacks
- Maintain conversation context efficiently
- Apply content filtering for safety
```

Requirements Validation Script (scripts/validate-requirements.js):
```javascript
const fs = require('fs');
const path = require('path');

class RequirementsValidator {
  constructor() {
    this.requirements = [];
    this.validationResults = [];
  }

  loadRequirements() {
    const requirementsDir = path.join(__dirname, '../docs/requirements');
    const files = fs.readdirSync(requirementsDir);

    files.forEach(file => {
      if (file.endsWith('.md')) {
        const content = fs.readFileSync(path.join(requirementsDir, file), 'utf8');
        this.parseRequirements(content, file);
      }
    });
  }

  parseRequirements(content, filename) {
    const requirementRegex = /## (FR-\d+): (.+)/g;
    let match;

    while ((match = requirementRegex.exec(content)) !== null) {
      const [, id, title] = match;
      const requirement = {
        id,
        title,
        filename,
        status: this.extractStatus(content, match.index),
        priority: this.extractPriority(content, match.index),
        acceptanceCriteria: this.extractAcceptanceCriteria(content, match.index)
      };

      this.requirements.push(requirement);
    }
  }

  extractStatus(content, startIndex) {
    const statusMatch = content.slice(startIndex).match(/\*\*Status\*\*: (.+)/);
    return statusMatch ? statusMatch[1] : 'Unknown';
  }

  extractPriority(content, startIndex) {
    const priorityMatch = content.slice(startIndex).match(/\*\*Priority\*\*: (.+)/);
    return priorityMatch ? priorityMatch[1] : 'Medium';
  }

  extractAcceptanceCriteria(content, startIndex) {
    const nextRequirement = content.indexOf('## FR-', startIndex + 1);
    const section = nextRequirement > -1 ?
      content.slice(startIndex, nextRequirement) :
      content.slice(startIndex);

    const criteria = [];
    const criteriaRegex = /- \[ \] (.+)/g;
    let match;

    while ((match = criteriaRegex.exec(section)) !== null) {
      criteria.push({
        description: match[1],
        completed: false
      });
    }

    return criteria;
  }

  validateImplementation() {
    this.requirements.forEach(req => {
      const validation = {
        requirementId: req.id,
        title: req.title,
        status: req.status,
        implementationStatus: this.checkImplementation(req),
        testCoverage: this.checkTestCoverage(req),
        recommendations: []
      };

      if (validation.implementationStatus === 'incomplete') {
        validation.recommendations.push('Implementation needs completion');
      }

      if (validation.testCoverage < 80) {
        validation.recommendations.push('Increase test coverage');
      }

      this.validationResults.push(validation);
    });
  }

  checkImplementation(requirement) {
    // Check if implementation exists based on requirement ID
    const implementationPaths = [
      `src/components/${requirement.id.toLowerCase()}`,
      `src/services/${requirement.id.toLowerCase()}`,
      `src/features/${requirement.id.toLowerCase()}`
    ];

    const hasImplementation = implementationPaths.some(path =>
      fs.existsSync(path + '.ts') || fs.existsSync(path + '.tsx')
    );

    return hasImplementation ? 'complete' : 'incomplete';
  }

  checkTestCoverage(requirement) {
    // Simulate test coverage check
    return Math.floor(Math.random() * 100);
  }

  generateReport() {
    const report = {
      summary: {
        totalRequirements: this.requirements.length,
        implemented: this.validationResults.filter(r => r.implementationStatus === 'complete').length,
        averageTestCoverage: this.validationResults.reduce((sum, r) => sum + r.testCoverage, 0) / this.validationResults.length
      },
      details: this.validationResults
    };

    fs.writeFileSync(
      path.join(__dirname, '../docs/requirements/validation-report.json'),
      JSON.stringify(report, null, 2)
    );

    console.log('Requirements Validation Report Generated');
    console.log(`Total Requirements: ${report.summary.totalRequirements}`);
    console.log(`Implemented: ${report.summary.implemented}`);
    console.log(`Average Test Coverage: ${report.summary.averageTestCoverage.toFixed(2)}%`);
  }

  run() {
    this.loadRequirements();
    this.validateImplementation();
    this.generateReport();
  }
}

const validator = new RequirementsValidator();
validator.run();
```
~~~

Step 4: Architecture Implementation

Implement the complete system architecture:

~~~
Create the foundational architecture components:

Core Types (src/types/index.ts):
```typescript
// User and Authentication Types
export interface User {
  id: string;
  email: string;
  name: string;
  avatar?: string;
  role: UserRole;
  preferences: UserPreferences;
  createdAt: Date;
  lastLoginAt?: Date;
}

export interface UserRole {
  id: string;
  name: string;
  permissions: Permission[];
}

export interface Permission {
  resource: string;
  actions: string[];
}

export interface UserPreferences {
  theme: 'light' | 'dark' | 'system';
  language: string;
  notifications: NotificationSettings;
  accessibility: AccessibilitySettings;
}

// Chat and AI Types
export interface ChatMessage {
  id: string;
  conversationId: string;
  content: string;
  type: 'user' | 'assistant' | 'system';
  timestamp: Date;
  metadata?: MessageMetadata;
}

export interface MessageMetadata {
  tokens?: number;
  model?: string;
  confidence?: number;
  sources?: string[];
}

export interface Conversation {
  id: string;
  userId: string;
  title: string;
  messages: ChatMessage[];
  createdAt: Date;
  updatedAt: Date;
  metadata: ConversationMetadata;
}

export interface ConversationMetadata {
  totalMessages: number;
  totalTokens: number;
  lastActivity: Date;
  tags: string[];
}

// API Response Types
export interface ApiResponse<T> {
  success: boolean;
  data?: T;
  error?: ApiError;
  metadata?: ResponseMetadata;
}

export interface ApiError {
  code: string;
  message: string;
  details?: Record<string, any>;
}

export interface ResponseMetadata {
  requestId: string;
  timestamp: Date;
  version: string;
}
```

Application Configuration (src/config/app.ts):
```typescript
export interface AppConfig {
  app: {
    name: string;
    version: string;
    environment: 'development' | 'staging' | 'production';
    baseUrl: string;
  };

  api: {
    baseUrl: string;
    timeout: number;
    retryAttempts: number;
    retryDelay: number;
  };

  auth: {
    tokenExpiry: number;
    refreshTokenExpiry: number;
    sessionTimeout: number;
    mfaRequired: boolean;
  };

  ai: {
    provider: 'bedrock';
    model: string;
    maxTokens: number;
    temperature: number;
    timeout: number;
  };

  features: {
    fileUpload: boolean;
    voiceInput: boolean;
    darkMode: boolean;
    analytics: boolean;
  };

  security: {
    cspEnabled: boolean;
    httpsOnly: boolean;
    secureHeaders: boolean;
    rateLimiting: boolean;
  };
}

export const appConfig: AppConfig = {
  app: {
    name: 'ChatBot App',
    version: process.env.VITE_APP_VERSION || '1.0.0',
    environment: (process.env.NODE_ENV as any) || 'development',
    baseUrl: process.env.VITE_APP_BASE_URL || 'http://localhost:3000'
  },

  api: {
    baseUrl: process.env.VITE_API_BASE_URL || 'http://localhost:3001/api',
    timeout: 30000,
    retryAttempts: 3,
    retryDelay: 1000
  },

  auth: {
    tokenExpiry: 15 * 60 * 1000, // 15 minutes
    refreshTokenExpiry: 7 * 24 * 60 * 60 * 1000, // 7 days
    sessionTimeout: 30 * 60 * 1000, // 30 minutes
    mfaRequired: process.env.VITE_MFA_REQUIRED === 'true'
  },

  ai: {
    provider: 'bedrock',
    model: process.env.VITE_AI_MODEL || 'anthropic.claude-3-sonnet-20240229-v1:0',
    maxTokens: 4000,
    temperature: 0.7,
    timeout: 30000
  },

  features: {
    fileUpload: process.env.VITE_FEATURE_FILE_UPLOAD === 'true',
    voiceInput: process.env.VITE_FEATURE_VOICE_INPUT === 'true',
    darkMode: true,
    analytics: process.env.VITE_FEATURE_ANALYTICS === 'true'
  },

  security: {
    cspEnabled: true,
    httpsOnly: process.env.NODE_ENV === 'production',
    secureHeaders: true,
    rateLimiting: true
  }
};
```

Error Handling System (src/utils/error-handler.ts):
```typescript
export class AppError extends Error {
  public readonly code: string;
  public readonly statusCode: number;
  public readonly isOperational: boolean;
  public readonly context?: Record<string, any>;

  constructor(
    message: string,
    code: string,
    statusCode: number = 500,
    isOperational: boolean = true,
    context?: Record<string, any>
  ) {
    super(message);
    this.code = code;
    this.statusCode = statusCode;
    this.isOperational = isOperational;
    this.context = context;

    Error.captureStackTrace(this, this.constructor);
  }
}

export class ValidationError extends AppError {
  constructor(message: string, context?: Record<string, any>) {
    super(message, 'VALIDATION_ERROR', 400, true, context);
  }
}

export class AuthenticationError extends AppError {
  constructor(message: string = 'Authentication failed', context?: Record<string, any>) {
    super(message, 'AUTHENTICATION_ERROR', 401, true, context);
  }
}

export class AuthorizationError extends AppError {
  constructor(message: string = 'Access denied', context?: Record<string, any>) {
    super(message, 'AUTHORIZATION_ERROR', 403, true, context);
  }
}

export class NotFoundError extends AppError {
  constructor(resource: string, context?: Record<string, any>) {
    super(`${resource} not found`, 'NOT_FOUND_ERROR', 404, true, context);
  }
}

export class RateLimitError extends AppError {
  constructor(message: string = 'Rate limit exceeded', context?: Record<string, any>) {
    super(message, 'RATE_LIMIT_ERROR', 429, true, context);
  }
}

export class ExternalServiceError extends AppError {
  constructor(service: string, message: string, context?: Record<string, any>) {
    super(`${service} service error: ${message}`, 'EXTERNAL_SERVICE_ERROR', 502, true, context);
  }
}

export const errorHandler = {
  handle: (error: Error): AppError => {
    if (error instanceof AppError) {
      return error;
    }

    // Handle specific error types
    if (error.name === 'ValidationError') {
      return new ValidationError(error.message);
    }

    if (error.name === 'UnauthorizedError') {
      return new AuthenticationError(error.message);
    }

    // Default to internal server error
    return new AppError(
      'An unexpected error occurred',
      'INTERNAL_SERVER_ERROR',
      500,
      false,
      { originalError: error.message, stack: error.stack }
    );
  },

  log: (error: AppError): void => {
    const logData = {
      message: error.message,
      code: error.code,
      statusCode: error.statusCode,
      context: error.context,
      stack: error.stack,
      timestamp: new Date().toISOString()
    };

    if (error.statusCode >= 500) {
      console.error('Server Error:', logData);
    } else {
      console.warn('Client Error:', logData);
    }
  }
};
```
~~~

Step 5: Foundation Validation and Testing

Implement comprehensive validation for the foundation:

~~~
Create foundation validation and testing:

Foundation Validation Script (scripts/validate-foundation.js):
```javascript
const fs = require('fs');
const path = require('path');
const { execSync } = require('child_process');

class FoundationValidator {
  constructor() {
    this.validationResults = {
      projectStructure: { passed: 0, failed: 0, details: [] },
      configuration: { passed: 0, failed: 0, details: [] },
      kiloIntegration: { passed: 0, failed: 0, details: [] },
      requirements: { passed: 0, failed: 0, details: [] },
      architecture: { passed: 0, failed: 0, details: [] }
    };
  }

  validateProjectStructure() {
    const requiredDirectories = [
      'src/components',
      'src/services',
      'src/utils',
      'src/types',
      'tests/unit',
      'tests/integration',
      'docs/api',
      'config/environments',
      '.kiro/settings',
      '.kiro/steering',
      '.kiro/hooks'
    ];

    const requiredFiles = [
      'package.json',
      'tsconfig.json',
      'README.md',
      '.gitignore',
      '.env.example'
    ];

    requiredDirectories.forEach(dir => {
      if (fs.existsSync(dir)) {
        this.validationResults.projectStructure.passed++;
        this.validationResults.projectStructure.details.push(`✅ Directory exists: ${dir}`);
      } else {
        this.validationResults.projectStructure.failed++;
        this.validationResults.projectStructure.details.push(`❌ Missing directory: ${dir}`);
      }
    });

    requiredFiles.forEach(file => {
      if (fs.existsSync(file)) {
        this.validationResults.projectStructure.passed++;
        this.validationResults.projectStructure.details.push(`✅ File exists: ${file}`);
      } else {
        this.validationResults.projectStructure.failed++;
        this.validationResults.projectStructure.details.push(`❌ Missing file: ${file}`);
      }
    });
  }

  validateConfiguration() {
    try {
      // Validate package.json
      const packageJson = JSON.parse(fs.readFileSync('package.json', 'utf8'));

      const requiredScripts = ['dev', 'build', 'test', 'lint', 'type-check'];
      requiredScripts.forEach(script => {
        if (packageJson.scripts && packageJson.scripts[script]) {
          this.validationResults.configuration.passed++;
          this.validationResults.configuration.details.push(`✅ Script configured: ${script}`);
        } else {
          this.validationResults.configuration.failed++;
          this.validationResults.configuration.details.push(`❌ Missing script: ${script}`);
        }
      });

      // Validate TypeScript configuration
      if (fs.existsSync('tsconfig.json')) {
        const tsConfig = JSON.parse(fs.readFileSync('tsconfig.json', 'utf8'));
        if (tsConfig.compilerOptions && tsConfig.compilerOptions.strict) {
          this.validationResults.configuration.passed++;
          this.validationResults.configuration.details.push('✅ TypeScript strict mode enabled');
        } else {
          this.validationResults.configuration.failed++;
          this.validationResults.configuration.details.push('❌ TypeScript strict mode not enabled');
        }
      }

    } catch (error) {
      this.validationResults.configuration.failed++;
      this.validationResults.configuration.details.push(`❌ Configuration validation error: ${error.message}`);
    }
  }

  validateKiroIntegration() {
    // Validate MCP configuration
    if (fs.existsSync('.kiro/settings/mcp.json')) {
      try {
        const mcpConfig = JSON.parse(fs.readFileSync('.kiro/settings/mcp.json', 'utf8'));
        if (mcpConfig.mcpServers) {
          this.validationResults.kiloIntegration.passed++;
          this.validationResults.kiloIntegration.details.push('✅ MCP servers configured');
        }
      } catch (error) {
        this.validationResults.kiloIntegration.failed++;
        this.validationResults.kiloIntegration.details.push('❌ Invalid MCP configuration');
      }
    } else {
      this.validationResults.kiloIntegration.failed++;
      this.validationResults.kiloIntegration.details.push('❌ MCP configuration missing');
    }

    // Validate Agent Steering
    const steeringFiles = fs.readdirSync('.kiro/steering').filter(f => f.endsWith('.md'));
    if (steeringFiles.length > 0) {
      this.validationResults.kiloIntegration.passed++;
      this.validationResults.kiloIntegration.details.push(`✅ Agent Steering configured (${steeringFiles.length} files)`);
    } else {
      this.validationResults.kiloIntegration.failed++;
      this.validationResults.kiloIntegration.details.push('❌ No Agent Steering files found');
    }

    // Validate Agent Hooks
    const hookFiles = fs.readdirSync('.kiro/hooks').filter(f => f.endsWith('.json'));
    if (hookFiles.length > 0) {
      this.validationResults.kiloIntegration.passed++;
      this.validationResults.kiloIntegration.details.push(`✅ Agent Hooks configured (${hookFiles.length} hooks)`);
    } else {
      this.validationResults.kiloIntegration.failed++;
      this.validationResults.kiloIntegration.details.push('❌ No Agent Hooks found');
    }
  }

  validateRequirements() {
    if (fs.existsSync('docs/requirements')) {
      const requirementFiles = fs.readdirSync('docs/requirements').filter(f => f.endsWith('.md'));
      if (requirementFiles.length > 0) {
        this.validationResults.requirements.passed++;
        this.validationResults.requirements.details.push(`✅ Requirements documented (${requirementFiles.length} files)`);
      } else {
        this.validationResults.requirements.failed++;
        this.validationResults.requirements.details.push('❌ No requirement files found');
      }
    } else {
      this.validationResults.requirements.failed++;
      this.validationResults.requirements.details.push('❌ Requirements directory missing');
    }

    // Check for validation script
    if (fs.existsSync('scripts/validate-requirements.js')) {
      this.validationResults.requirements.passed++;
      this.validationResults.requirements.details.push('✅ Requirements validation script exists');
    } else {
      this.validationResults.requirements.failed++;
      this.validationResults.requirements.details.push('❌ Requirements validation script missing');
    }
  }

  validateArchitecture() {
    // Check core architecture files
    const architectureFiles = [
      'src/types/index.ts',
      'src/config/app.ts',
      'src/utils/error-handler.ts'
    ];

    architectureFiles.forEach(file => {
      if (fs.existsSync(file)) {
        this.validationResults.architecture.passed++;
        this.validationResults.architecture.details.push(`✅ Architecture file exists: ${file}`);
      } else {
        this.validationResults.architecture.failed++;
        this.validationResults.architecture.details.push(`❌ Missing architecture file: ${file}`);
      }
    });

    // Validate TypeScript compilation
    try {
      execSync('npm run type-check', { stdio: 'pipe' });
      this.validationResults.architecture.passed++;
      this.validationResults.architecture.details.push('✅ TypeScript compilation successful');
    } catch (error) {
      this.validationResults.architecture.failed++;
      this.validationResults.architecture.details.push('❌ TypeScript compilation failed');
    }
  }

  generateReport() {
    const totalPassed = Object.values(this.validationResults).reduce((sum, category) => sum + category.passed, 0);
    const totalFailed = Object.values(this.validationResults).reduce((sum, category) => sum + category.failed, 0);
    const totalTests = totalPassed + totalFailed;
    const successRate = totalTests > 0 ? (totalPassed / totalTests * 100).toFixed(2) : 0;

    console.log('\n📋  Foundation Validation Report');
    console.log('================================');
    console.log(`Overall Success Rate: ${successRate}% (${totalPassed}/${totalTests})`);
    console.log('');

    Object.entries(this.validationResults).forEach(([category, results]) => {
      const categorySuccess = results.passed + results.failed > 0 ?
        (results.passed / (results.passed + results.failed) * 100).toFixed(2) : 0;

      console.log(`${category.toUpperCase()}: ${categorySuccess}% (${results.passed}/${results.passed + results.failed})`);
      results.details.forEach(detail => console.log(`  ${detail}`));
      console.log('');
    });

    // Save detailed report
    const report = {
      timestamp: new Date().toISOString(),
      summary: {
        totalTests,
        totalPassed,
        totalFailed,
        successRate: parseFloat(successRate)
      },
      categories: this.validationResults
    };

    fs.writeFileSync('docs/foundation-validation-report.json', JSON.stringify(report, null, 2));
    console.log('📊 Detailed report saved to docs/foundation-validation-report.json');
  }

  run() {
    console.log('🚀 Starting Foundation Validation...\n');

    this.validateProjectStructure();
    this.validateConfiguration();
    this.validateKiroIntegration();
    this.validateRequirements();
    this.validateArchitecture();

    this.generateReport();
  }
}

const validator = new FoundationValidator();
validator.run();
```

Foundation Test Suite (tests/foundation/foundation.test.ts):
```typescript
import { describe, it, expect, beforeAll } from '@jest/globals';
import fs from 'fs';
import path from 'path';
import { appConfig } from '../../src/config/app';
import { AppError, errorHandler } from '../../src/utils/error-handler';

describe('Foundation Implementation', () => {
  describe('Project Structure', () => {
    it('should have all required directories', () => {
      const requiredDirs = [
        'src/components',
        'src/services',
        'src/utils',
        'src/types',
        'tests/unit',
        'tests/integration',
        'docs/api',
        'config/environments'
      ];

      requiredDirs.forEach(dir => {
        expect(fs.existsSync(dir)).toBe(true);
      });
    });

    it('should have all required configuration files', () => {
      const requiredFiles = [
        'package.json',
        'tsconfig.json',
        'README.md',
        '.gitignore',
        '.env.example'
      ];

      requiredFiles.forEach(file => {
        expect(fs.existsSync(file)).toBe(true);
      });
    });
  });

  describe('Configuration', () => {
    it('should have valid app configuration', () => {
      expect(appConfig).toBeDefined();
      expect(appConfig.app.name).toBeDefined();
      expect(appConfig.app.version).toBeDefined();
      expect(appConfig.api.baseUrl).toBeDefined();
      expect(appConfig.auth.tokenExpiry).toBeGreaterThan(0);
    });

    it('should have proper environment-specific settings', () => {
      expect(['development', 'staging', 'production']).toContain(appConfig.app.environment);

      if (appConfig.app.environment === 'production') {
        expect(appConfig.security.httpsOnly).toBe(true);
        expect(appConfig.security.cspEnabled).toBe(true);
      }
    });
  });

  describe('Error Handling', () => {
    it('should create proper error instances', () => {
      const error = new AppError('Test error', 'TEST_ERROR', 400);

      expect(error.message).toBe('Test error');
      expect(error.code).toBe('TEST_ERROR');
      expect(error.statusCode).toBe(400);
      expect(error.isOperational).toBe(true);
    });

    it('should handle unknown errors properly', () => {
      const unknownError = new Error('Unknown error');
      const handledError = errorHandler.handle(unknownError);

      expect(handledError).toBeInstanceOf(AppError);
      expect(handledError.code).toBe('INTERNAL_SERVER_ERROR');
      expect(handledError.statusCode).toBe(500);
    });
  });

  describe('Kiro Integration', () => {
    it('should have MCP configuration', () => {
      const mcpConfigPath = '.kiro/settings/mcp.json';
      expect(fs.existsSync(mcpConfigPath)).toBe(true);

      const mcpConfig = JSON.parse(fs.readFileSync(mcpConfigPath, 'utf8'));
      expect(mcpConfig.mcpServers).toBeDefined();
      expect(Object.keys(mcpConfig.mcpServers).length).toBeGreaterThan(0);
    });

    it('should have Agent Steering files', () => {
      const steeringDir = '.kiro/steering';
      expect(fs.existsSync(steeringDir)).toBe(true);

      const steeringFiles = fs.readdirSync(steeringDir).filter(f => f.endsWith('.md'));
      expect(steeringFiles.length).toBeGreaterThan(0);
    });

    it('should have Agent Hooks configured', () => {
      const hooksDir = '.kiro/hooks';
      expect(fs.existsSync(hooksDir)).toBe(true);

      const hookFiles = fs.readdirSync(hooksDir).filter(f => f.endsWith('.json'));
      expect(hookFiles.length).toBeGreaterThan(0);
    });
  });
});
```

Run foundation validation:
```bash
# Install dependencies
npm install

# Run foundation validation
node scripts/validate-foundation.js

# Run foundation tests
npm test tests/foundation

# Generate documentation
npm run docs:generate
```
~~~

Foundation Implementation Complete

Your Foundation phase implementation is now complete with:

✅ Implemented Components

- **Complete project structure** with professional organization
- **Advanced Kiro integration** with MCP servers, Agent Hooks, and Agent Steering
- **Comprehensive configuration** with TypeScript, ESLint, and build tools
- **Requirements validation** with automated tracking and reporting
- **System architecture** with error handling, configuration management, and type safety

✅ Quality Assurance

- **Automated validation** scripts for continuous quality checking
- **Comprehensive test suite** covering all foundation components
- **Documentation generation** with TypeDoc and markdown
- **Code quality enforcement** with ESLint, Prettier, and TypeScript strict mode

✅ Kiro Features

- **MCP Server Integration**: AWS docs, GitHub, web search, npm registry
- **Agent Hooks**: Automated code quality validation and test execution
- **Agent Steering**: Project standards and security requirements enforcement
- **Living Documentation**: Requirements tracking with validation reports

aws workshop studio ⚙ rajeshkumar ▼

Event ends in 2 days 8 hours 10 minutes. ✕

1. **Proceed to Infrastructure Implementation** - Set up AWS resources and deployment pipeline
2. **Begin Frontend Development** - Build React components with the established architecture
3. **Implement Backend Services** - Create APIs and business logic with the foundation patterns
4. **Apply Security Controls** - Layer security throughout the application

Your foundation provides a solid, professional base for building a production-ready chatbot application with specification-driven development and advanced Kiro features.

Previous Next

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences